Distributed Motion Planning for Navigation in Closed Formation Among Obstacles

Szilárd Novoth, Tamás Péni, Roland Tóth

***Abstract*— This paper introduces a distributed strategy for motion planning for a formation of agents. The method navigates the group through a complex environment, tracks a reference trajectory and avoids collision with obstacles while preserving the formation and only allowing dynamic shrinking, expansion or rotation of the formation. The problem is solved using a novel hierarchical optimization scheme. The lower level algorithm generates way-points, which restrict the solution space of the higher level algorithm, allowing it to find a solution in a faster and more reliable manner. The higher level algorithm formulates the problem as an optimization problem in the moving Serret-Frenet coordinate frame. The method is implemented in 2D for simplicity but can be readily extended to 3D and as part of a higher level navigation system. The use of B-splines results in smooth and dynamically feasible trajectories. All the computations are solved in real time in a distributed manner. The distribution of the optimization problem is done via the Alternating Direction Method of Multipliers among the vehicles, leading to a fast, scalable algorithm.**

***Index Terms*— Multi-agent systems, distributed motion planning, trajectory optimization, obstacle avoidance, Serret-Frenet frame**

I. Introduction

Improvements in computational power and communication technologies inevitably lead to increased attention towards multi-vehicle systems (MVS). Important areas, where MVS can excel in comparison to systems consisting of a single vehicle, are cooperative transportation, inspection and surveillance. The systems, where autonomous vehicles move in a predefined formation can accomplish these tasks with better efficiency.

Formation control of MVS includes navigating a formation through a complex environment. The group may be guided by a reference path, required to hold a strict formation, allowed to dynamically scale and rotate, or designed to navigate in a loose formation. Reference tracking, obstacle avoidance and formation keeping pose a complex problem, where often conflicting aspects of design requirements have to be met.

In [1] the path following problem was considered on a unicycle-model. The dynamics was formulated in a moving Serret-Frenet frame which was regarded as a "virtual target vehicle". A trajectory control scheme was considered in [2] for only a single vehicle. In [3] a formation control problem was examined and a group of agents were guided along a path but no trajectory tracking was considered.

Formation control strategies in general can be divided into three main categories: behavioral-based methods [4], leader-follower approaches [5] and virtual-structure methods [6]. These are well established methods for maintaining a given formation while moving on a predefined path. In these cases the path is generated offline and broadcasted to the agents. This method however is not robust against the failure of the central computational unit.

A distributed approach to formation control was considered in [7], where ADMM was applied to distribute the optimal control problem among many agents to provide a fully decentralised framework. Similarly, [8] used ADMM to formulate a distributed trajectory tracking flocking control method. The solution guarantees inter-agent collision avoidance, but no strict formation definition can be prescribed. Exact formation definition can be assigned to a group of agents using a distributed control scheme presented in [9]. In the paper a formation of vehicles travels through a dynamic environment while avoiding collisions with moving obstacles. Avoiding obstacles however may cause the formation to break. In [10] a similar scenario was described but the formation constraints were relaxed enabling the formation to grow or contract dynamically, rather than to distort, while avoiding obstacles.

A navigation method, which accepts guidance from a reference trajectory and allows dynamic scaling and rotation of the group proves to be useful in various industrial applications. In some transportation or aerial imaging scenarios the general direction of the vehicles is more important than their distance. By maintaining the overall shape, equal load can be guaranteed on aerial transportation vehicles, thereby ensuring no tilting of the package and even drainage of battery. Similarly, when directional antennas are used to communicate between agents, formation keeping is crucial to stay connected.

The novelty of our approach lies in providing a distributed, real-time framework for an important formation control problem. The proposed method, solves the problem of navigating a formation of vehicles through a complex environment, guided by a reference trajectory. Obstacle avoidance is performed by scaling or rotating the formation, rather than distorting its shape. Our contributions are as follows:

1) We propose a hierarchical optimization scheme, where the lower level algorithm generates way-points, that restrict the solution space of the higher level algorithm, thus leading to faster and more reliable trajectory

The research was supported by the Ministry of Innovation and Technology NRDI Office within the framework of the Autonomous Systems National Laboratory Program.

S. Novoth, T. Péni and R. Tóth are with the Systems and Control Laboratory of Institute for Computer Science and Control (SZTAKI), Eötvös Lóránd Research Network (ELKH), H-1111 Kende u. 13-17, Budapest, Hungary (email: novothsz@gmail.com, peni@sztaki.hu, toth.roland@sztaki.hu)

generation.
2) We employ a distributed nonlinear optimization program, which is scalable and can provide vehicle trajectories in real time.
3) We formulate the entire trajectory planning algorithm in a translating Serret-Frenet coordinate frame, that can be regarded as an air-corridor or road through a cluttered environment.
4) We present an obstacle avoidance framework, capable of maintaining the shape of the prescribed formation even in a cluttered environment. The formation is able to automatically shrink, grow or rotate to avoid collision with obstacles.

The work is structured as follows: Section II briefly defines the problem. Section III-A derives the kinematics of the vehicle in the moving Serret-Frenet coordinate system. Section III-B explains the intuition behind the proposed hierarchical optimization scheme. Section III-C proposes an effective method to generate way-points in a complex environment. Section III-D introduces B-spline parametrisation and Section III-E formulates the problem as a centralised optimization problem. Section III-F distributes the problem using the Alternating Direction Method of Multipliers (ADMM), followed by Section III-G, which presents a successive trajectory generation scheme for real-time implementation. Section IV provides simulation results and lastly, Section V gives a brief conclusion.

II. Problem formulation

In this paper we aim at solving a trajectory tracking problem in a distributed manner with a group of vehicles, which have to keep a predefined formation and avoid collision with static obstacles.

For the sake of generality, the formation is an arbitrary polytope, whose vertices $\nu_\ell \in \mathbb{R}^2;\ \ell \in \mathbb{I}_0^{N_\ell}$ impose relative positions $x^{\mathrm{ref}} \in \mathbb{R}^2$ on the vehicles. These relative positions must be kept during the entire maneuver. To avoid obstacles, two transformations are permitted on the polytope: rotation and scaling, but no distortion. Limits on the shrinking and expansion of the formation are also considered to represent physical limitations. In this work, trajectory tracking will be imposed by requiring the formation center to track a trajectory $r_{\mathrm{f}}(t) \in \mathbb{R}^2$. The formation center is defined as the mean of its vertecies. Consequently, because to each of the vertex there is an associated vehicle, the mean of the vehicle positions should also be equal to $r_{\mathrm{f}}(t)$ at all times during the maneuver.

The group of vehicles need to design their own trajectory separately, but are allowed to communicate with each other, for which no communication delay is considered (e.g, wireless 5G connection). The trajectories must be generated in real-time and adhere to obstacle avoidance constraints and input constraints, imposed by the physical limitations of the system. Once calculated, trajectories are forwarded to the real-world vehicles, whose real-time controller is assumed to be able to perfectly track them. It is also assumed, that each vehicle runs the same exact code and detects the same obstacles.

III. Proposed method

To solve the outlined problem, we are proposing a unique approach, whereby trajectory tracking and distributed formation control is performed in a moving coordinate frame, rather than in an inertial frame. In fact, the trajectory to be tracked, namely $r_{\mathrm{f}}(t)$ will form this moving coordinate frame. This choice leads to a simple formulation of the problem and let's the group to be guided through a complex terrain. The problem will be solved using a novel hierarchical optimization scheme. A proposed lower level algorithm generates way-points for the group, which encode feasible formation configuration when maneuvering to avoid obstacles. The higher level algorithm solves an optimal control problem, whereas state variables of the vehicles, the cost function and all the constraints are formulated in the moving coordinate system. The problem is solved using numerical optimization, whose solution space is restricted by the solution of the lower level algorithm.

To describe our proposed method, we will proceed by systematically developing the building blocks of the entire trajectory planning algorithm.

A. System model in the moving coordinate frame

In trajectory tracking, a vehicle has to follow a time-parameterised reference trajectory, which can arise from the need to stay inside a prescribed air-corridor or to stay on a designated road. An efficient way to formulate the planning along this route is to express the vehicle movement in the Serret-Frenet frame $\mathcal{F}$, centered at $r_{\mathrm{f}}(t) \in \mathbb{R}^2$, where $t \in [0, T] \subset \mathbb{R}$. This moving coordinate frame has to be tracked by the group of vehicles, while holding a predefined formation.

Figure 1 shows a vehicle, whose position is defined in the moving Frenet frame. In the Figure $\mathcal{I}$ represents the inertial frame, $\mathcal{F}$ represents the Frenet frame and θ the rotation of $\mathcal{F}$ w.r.t $\mathcal{I}$. The position of the vehicle in $\mathcal{F}$ is given by $p(t), q(t) \in \mathbb{R}$, whereas $v_{\mathrm{x}}(t), v_{\mathrm{y}}(t) \in \mathbb{R}$ stand for its velocity in the world-coordinate frame $\mathcal{I}$. The symbols $s(t), v_{\mathrm{s}}(t), a_{\mathrm{s}}(t)$ represent the distance, velocity and acceleration of $\mathcal{F}$ along the path at time t.

For the sake of simplicity, a double integrator model is used to represent the lateral and longitudinal dynamics of the vehicle, whose physical limitations can be approximated by constraining the velocity and acceleration of the simplified model. Thus, using the formulas presented in [1] the equations of motion can be defined as:

$$\begin{bmatrix} \dot{p} \\ \dot{q} \\ \dot{v}_{\mathrm{x}} \\ \dot{v}_{\mathrm{y}} \end{bmatrix} = \begin{bmatrix} \cos(\theta)v_{\mathrm{x}} + \sin(\theta)v_{\mathrm{y}} - v_{\mathrm{s}}(1 - qc(s)) \\ -\sin(\theta)v_{\mathrm{x}} + \cos(\theta)v_{\mathrm{y}} - v_{\mathrm{s}}pc(s) \\ a_{\mathrm{x}} \\ a_{\mathrm{y}} \end{bmatrix}, \quad (1)$$

where $a_{\mathrm{x}}, a_{\mathrm{y}}$ stand for the acceleration of the vehicle in the world coordinate frame $\mathcal{I}$ and $c(s)$ represents the curvature of the reference trajectory at s.

B. Hierarchical optimization

Trajectory optimization has been proven to be capable of generating feasible trajectories for vehicles, even in complex, dynamic environments. One of the most recent and prominent algorithm is provided by the OMG-tools, presented in [11]. The algorithm is extended in [9] and applied on a MVS moving in a formation. However, when the group has to avoid obstacles, the formation breaks.

Closed formation keeping of MVS in a cluttered environment is a complex optimization problem. The decision of one vehicle avoiding an obstacle from a specific direction effects the trajectories of all the other vehicles in the group. Finding the global minima of the problem is computationally expensive and not guaranteed.

In this work we propose a hierarchical optimization scheme, where the higher level algorithm incorporates the solution of the lower level algorithm. The goal of the hierarchical optimization scheme is to improve solution time and increase reliability of entire motion planning procedure by restricting the solution space of the higher level algorithm, using the solution genereated by the lower level algorithm. Our method relies on a trajectory optimization scheme as a higher level algorithm, presented in Section III-E and Section III-F, which generates trajectories for the vehicles for a horizon T_h. Its solution space however is restricted by the way-points, generated by the lower level algorithm, presented in Section III-C. The method generates way-points for the same horizon T_h at time instances t_k, which serve as feasible formation configurations at the specific time instance. The lower level algorithm is called every time before the execution of the higher level algorithm. Meaning, in case of a receding horizon method, the lower level algorithm is called in every iteration for the planning horizon. Thus, if the environment changes, new way-points are generated to accommodate this change.

C. Dynamic formation generator

Finding an optimal path around obstacles is a hard optimization problem. The problem becomes even harder, when a formation constraint is present. Each vehicle's trajectory is dependent on all the other trajectories as well, and it is not straightforward, from which direction the given vehicle should avoid the obstacle to ensure, that the formation can be maintained.

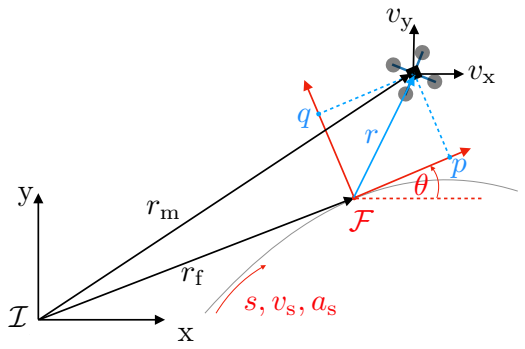


Fig. 1: Parameter definitions for a single agent and the moving coordinate frame.

Hereby we propose an efficient lower level algorithm, which calculates a series of $k \in \mathbb{Z}^+$ feasible formations along the Frenet frame. These intermediate formations are defined by way-points $x_{i,k}^{\text{way}} \in \mathbb{R}^2$, whereas $i \in \mathbb{I}_0^{N-1}$ indicates the ith vehicle, with $\mathbb{I}_{\tau_1}^{\tau_2}$ being defined as the index set $\{\tau \in \mathbb{Z} \mid \tau_1 \leq \tau \leq \tau_2\}$ and N indicating the number of vehicles in the group. Way-points are given w.r.t. the Frenet frame $\mathcal{F}$ and each vehicle has to pass through its assigned way-point. These intermediate points restrict the solution space of the higher level optimization algorithm and act as guidance on how to rotate and scale the shape to best avoid the obstacles along the path.

The proposed algorithm, called Dynamic Formation Generator (DFG), generates feasible formations along $\mathcal{F}$ for a horizon T_h, at time instances t_k. Assuming, that the same DFG algorithm is running on the vehicles and each of them are aware of the same exact obstacles, the DFG algorithm can be executed on them separately and will output the same solution.

DFG is an event-based algorithm, which generates feasible formations in three distinct scenarios: 1) when an obstacles enters the collision radious, 2) when $T_{\text{free}} \in [0, T_h]$ time has passed and no collision was detected since t_k and 3) at the end of the horizon T_h. Thus, at least one set of way-points are generated by DFG.

1) Obstacle-triggered way-point generation: In the case, when way-point generation is triggered by an obstacle, the following steps are performed.

First, a collision zone around the formation is defined by a circle, with center point $r_{\text{f}}(t)$ and radius $r_k^{\text{col}} \in \mathcal{R}$, defined as $r_k^{\text{col}} = max\|x_{i,k-1}^{\text{way}} + r_i^{\text{veh}}\|_2 \ \forall i$, where $x_{i,k-1}^{\text{way}}$ is the previously generated way-point by DFG (or the starting position for the given vehicle for $k = 0$). Because the formation center should always stay at $r_{\text{f}}(t)$, r_k^{col} is the maximum distance of any formation corner from the Frenet frame. When an obstacle enters the collision zone at t_k^{in}, its movement is tracked until it leaves the zone at t_k^{out}. Because DFG only plans until T_h, if the obstacle remains in the collision zone beyond T_h, t_k^{out} will be capped at T_h. Then, DFG will attempt to generate a feasible formation for the time instance $t_k = \frac{t_k^{\text{in}} + t_k^{\text{out}}}{2}$, one, which is also feasible for $t \in [t_k^{\text{in}}, t_k^{\text{out}}]$. It is assumed, that both the trajectory of the Frenet frame and the trajectory of the obstacle is known at least until t_k^{out}.

Second, formation candidates are generated by DFG by rotating and scaling the previously calculated formation (or the starting formation at $k = 0$) w.r.t $r_{\text{f}}(t_k)$. Rotation with angle $\phi, \{\phi \in \mathbb{R} \mid \phi_{\text{lb}} \leq \phi \leq \phi_{\text{ub}}\}$ and scaling with $h, \{h \in \mathbb{R} \mid h_{\text{lb}} \leq h \leq h_{\text{ub}}\}$ happens at N_ϕ and N_h linearly spaced intervals respectively, resulting in $\text{N}_{\phi h} = \text{N}_\phi \cdot \text{N}_h$ formation candidates. Reasonable upper and lower bounds for the rotation angle are $[\phi_{\text{lb}}, \phi_{\text{ub}}] = [-\pi/2, \pi/2]$. For h, the lower bound h_{lb} should reflect how close we allow the vehicles to get, and the upper bound h_{ub} should be a sensible value, encoding the maximum allowed distance

from the Frenet frame. Values N_ϕ and N_h effect how fast the algorithm runs, hence should be determined carefully. Setting $\mathrm{N}_\phi = 20$ lets DFG choose from a fine set of rotation values, whereas by setting $\mathrm{N}_h = 7$, only a few differently scaled formation candidates will be generated.

Third, collision is checked for each formation candidate at $N_t = 10$ linearly spaced timesteps $t_{k,c}, \{t_{k,c} \in \mathbb{R} \mid t_k^{\mathrm{in}} \leq t_{k,c} \leq t_k^{\mathrm{out}}\}$. Candidates, where collision is detected are discarded by DFG.

Fourth, DFG picks the optimal formation from the generated formations, using their associated cost J^{DFG}. The cost of the formation candidates is evaluated based on the required rotation, scaling and the distance of the corresponding corners of the previous and the new formation, as defined in (2):

$$J_{k,n}^{\mathrm{DFG}} = \alpha_\phi \phi_{k,n}^2 + \alpha_{\mathrm{h}} h_{k,n}^2 + \frac{1}{N} \sum_{i=0}^{\mathrm{N}-1} \alpha_{\mathrm{d}} \| x_{i,k,n}^{\mathrm{way}} - x_{i,k-1}^{\mathrm{way}} \|_2, \tag{2}$$

where $J_{k,n}^{\mathrm{DFG}}$ is the cost of the nth formation candidate at the kth itertaion, with $n \in \mathbb{I}_0^{N_{\phi h}}$. Rotation, scaling and distance weights are represented by $\alpha_\phi, \alpha_{\mathrm{h}}, \alpha_{\mathrm{d}}$ respectively. The values $\phi_{k,n}, h_{k,n}, x_{i,k,n}^{\mathrm{way}}$ represent the rotation angle, scaling factor and the ith way-point of the nth formation candidate at iteration k. By appropriately choosing the weights, DFG's preference for rotation over scaling can be adjusted.

At times, DFG may fail to generate feasible way-points, because the environment is too heavily occluded, and all formation candidates would cause collision with obstacles for $t \in [t_k^{\mathrm{in}}, t_k^{\mathrm{out}}]$. In this scenario, DFG tries to generate two separate sets of way-points. One for $t \in [t_k^{\mathrm{in}}, t_k]$ and another for $t \in [t_k, t_k^{\mathrm{out}}]$. If still no feasible formations can be found, no way-points are generated for t_k. Consequently, the solution space of the higher level algorithm will not be reduced, allowing it to find feasible solution without the guidence of DFG.

2) Free-time triggered way-point generation: It is desired, that the formation returns to its original configuration. For this reason, if $T_{\mathrm{free}} \in [0, T_h]$ time has passed and no collision was detected since t_k, DFG generates way-points, resembling the original formation for $t_k + T_{\mathrm{free}}$. Thus the initial state of the formation is restored.

3) Horizon-triggered way-point generation: DFG is employed to generate feasible final positions for the vehicles at the end of the horizon, resembling the prescribed formation. If the presence of an obstacle is preventing it, that means, that an obstacle-triggered way-point has already been generated, which is also feasible for the final position.

D. Spline parametrization

After determining the way-points, the next step is constructing a feasible trajectory for each vehicle to access the way-points. By parameterizing the trajectories by splines, a relaxed optimization problem can be defined, with a limited number of decision variables.

A B-spline $\hat{y}(t) \in \mathbb{R}$ is defined by its basis functions and coefficients as:

$$\hat{y}(t) = \sum_{m=0}^{M-1} \vartheta_{\mathrm{y},m} b_m(t) = \vartheta_{\mathrm{y}}^\top b(t), \tag{3}$$

where $b(t) = [b_0(t)\ b_1(t)\ \dots\ b_{M-1}(t)] \in \mathbb{R}^{M-1}$ are the B-spline basis. The B-spline coefficients of the trajectory $\hat{y}(t)$ are denoted by the vector: $\vartheta^\top = [\vartheta_{\mathrm{y},0}\ \vartheta_{\mathrm{y},1}\ \dots\ \vartheta_{\mathrm{y},M-1}]^\top \in \mathbb{R}^{M-1}$.

As described in [12], B-spline parameterization has major benefits in robotic applications. The stable evaluation of their integral and derivative is useful when defining cost functions and system equations. Furthermore, B-splines are always contained in the convex hull of their coefficients. This property allows us to define bounds on the spline by enforcing the constraint on its coefficients:

$$\hat{y}(t) \geq 0 \Leftarrow \mathrm{y} \geq 0, \forall t \in [0, T]. \tag{4}$$

Furthermore, any polynomial function of a spline is a spline as well, thus polynomial constraints on spline trajectories can also be relaxed.

For a concise definition of the basis functions and the derivatives of B-splines, the reader is kindly referred to [13].

E. Centralised trajectory optimization problem

In this work, we search for state trajectories $\hat{x}_i(t) = [\hat{p}_i(t)\ \hat{q}_i(t)]^\top$ for N number of vehicles, that bring the dynamical system from its starting state to its desired final state. The trajectories should minimize the cost function $J_i(\cdot)$ and adhere to starting position, final position, dynamics and collision avoidance constraints, defined by $\mathcal{X}^i$. The index $i \in \mathbb{I}_0^{N-1}$ indicates the ith vehicle, and the neighbourhood set of vehicle i is denoted by $\mathcal{N}_i$. This set includes all vehicles $j \neq i$ with which i communicates and towards which a formation constraint is defined. Similarly to i, the state trajectories of vehicle j are given by $\hat{x}_j(t)$. Formation constraint for vehicle i is prescribed by $g_i^{\mathrm{form}}(\cdot)$, trajectory tracking constraint is defined by $g_i^{\mathrm{track}}(\cdot)$ and way-point crossing constraint is defined by $g_i^{\mathrm{way}}(\cdot)$. The trajectory $\hat{\phi}(t) \in \mathbb{R}$ describes the rotation of the formation w.r.t. $r_{\mathrm{f}}(t)$. Formation and trajectory tracking constraints are enforced for $t \in [0, T_h]$, while way-point constraints for all t_k values. Minimisation is performed on the combined spline parameters $\vartheta = [\vartheta_{\mathrm{x}}^\top\ \vartheta_\phi^\top\ \vartheta_{\mathrm{a}}^\top\ \vartheta_{\mathrm{b}}^\top]^\top$, where ϑ_{x} is a vector composed of the coefficients of $\hat{x}_i(t); \forall i \in \mathbb{I}_0^{N-1}$ and ϑ_ϕ represents the coefficient vector of $\hat{\phi}(t)$. Similarly, $\vartheta_{\mathrm{a}}, \vartheta_{\mathrm{b}}$ are the combined vector representation of the coefficients of $\hat{a}_{i,w}(t), \hat{b}_{i,w}(t); \forall i \in \mathbb{I}_0^{N-1}; \forall w \in \mathbb{I}_0^{N_w - 1}$, defined in Section III-E.5. Thus, the centralised optimization problem becomes:

$$
\begin{aligned}
\underset{\vartheta}{\text{minimise}} \quad & \frac{1}{N}\sum_{i=0}^{N-1} J_i(\ddot{\hat{x}}_i(t)) \\
\text{subject to} \quad & \hat{x}_i(t) \in \mathcal{X}_i \\
& g_i^{\text{form}}(\hat{x}_i(t), \hat{x}_j(t), \hat{\phi}(t)) = 0 \\
& g_i^{\text{track}}(\hat{x}_i(t), \hat{x}_j(t)) = 0 \\
& g_i^{\text{way}}(\hat{x}_i(t_k)) \geq 0; \qquad \forall t_k \\
& \forall i \in \mathbb{I}_0^{N-1}; \;\; \forall j \in \mathcal{N}_i; \;\; \forall t \in [0, T_h].
\end{aligned} \tag{5}
$$

1) Objective function: The objective function (6) is the integral of an $\mathcal{L}_2([0, T_h])$ norm, multiplied by the acceleration penalty cost $\alpha_{\text{a}} \in \mathbb{R}$. It is designed to minimise the overall extra power the vehicles need to invest to follow $r_{\text{f}}(t)$.

$$
J_i(\ddot{\hat{x}}_i(t)) = \int_0^{T_h} \alpha_{\text{a}} \|\ddot{\hat{x}}_i(t)\|_2^2 \, dt, \tag{6}
$$

2) Formation constraint: To this end, we propose a novel formation constraint, which enables a group of vehicles to shrink, grow or rotate. The constraint enforces relative directions between the vehicles. The relative direction between vehicles i and j is defined by the reference vector $x_{i,j}^{\text{ref}} \in \mathbb{R}^2$, which connects their corresponding corners ν_ℓ of the reference polytope. By taking the cross product of the current relative position of the vehicles with the reference relative vector, an equality constraint can be formulated. By requiring the cross product to be zero, the two vectors are forced to be parallel with each other. Consequently, the constraint is able to maintain the formation of the shape, while allowing dynamic scaling. To also allow the rotation of the formation, the reference vector is multiplied by the rotation matrix $\mathbf{R}(\phi)$, as defined in (7).

$$
\mathbf{R}(\phi) = \begin{bmatrix} \cos(\phi) & -\sin(\phi) \\ \sin(\phi) & \cos(\phi) \end{bmatrix}. \tag{7}
$$

Thus, the formation constraint can be written as:

$$
\begin{aligned}
g_i^{\text{form}}(\hat{x}_i(t), \hat{x}_j(t), \hat{\phi}(t)) = \\
(\hat{x}_i(t) - \hat{x}_j(t)) \times (\mathbf{R}(\hat{\phi}(t)) \cdot x_{i,j}^{\text{ref}}).
\end{aligned} \tag{8}
$$

3) Trajectory tracking: Trajectory tracking is not performed by the individual vehicles, but by the formation as a whole. By requiring the mean of the vehicle positions to be zero (in the Frenet frame), we constrain the formation center to also be zero and hence to track the trajectory $r_{\text{f}}(t)$ during the maneuver. Consequently, the trajectory tracking is performed by imposing constraint (9) on the group:

$$
g_i^{\text{track}}(\hat{x}_i(t)) = \sum_{i=0}^{N-1} \hat{x}_i(t). \tag{9}
$$

4) Way-point crossing: In Section III-C the DFG algorithm was introduced, which quickly generates feasible way-points for the formation to pass through. These way-points serve as a guidance for the optimizer and provide useful information about how to turn or rotate the formation. To this end, we leverage this information by defining constraint (10). So as to not over-restrict the problem, a small user-defined value allows some deviation from the way-points. In this work it has been chosen to be the radius of the ith vehicle $r_i^{\text{veh}} \in \mathbb{R}$. This formulation leaves freedom for the optimizer to smoothly transform the formation, but still being guided by the feasible way-points. As a result, the way-point crossing constraint can be expressed as:

$$
g_i^{\text{way}}(\hat{x}_i(t_k)) = r_i^{\text{veh}} - \|\hat{x}_i(t_k) - x_{i,k}^{\text{way}}\|_2^2; \quad \forall k \in \mathbb{I}_0^{N_k - 1}. \tag{10}
$$

5) Obstacle avoidance: We propose to formulate the collision avoidance constraints in the Frenet frame, due to the already available state trajectories $\hat{x}_i(t)$. In the case of a static obstacle in a non-moving coordinate system, the corners of the obstacle remain constant i.e. don't depend on t. However, an additional challenge of a moving coordinate system is, that stationary obstacles become dynamic obstacles w.r.t. to the moving frame. This phenomena can be observed in Figure 2, which shows how a particular stationary object is perceived by a vehicle in the moving Frenet frame. Collision avoidance constraints must be satisfied for all obstacles $w \in \mathbb{I}_0^{N_w - 1}$, where N_w denotes the number of obstacles in the environment.

Obstacle avoidance can be formulated as a constraint using the separating hyperplane theorem [14]. The theorem states, that if two non-empty sets $\mathbb{S}_1$ and $\mathbb{S}_2$ are disjoint and convex, then there exists a hyperplane $\mathcal{H}\{x \in \mathbb{R}^n \mid a^\top x = b\}$ which separates the two sets. In such cases $\mathbb{S}_1$ will be on one side of the hyperplane $\{a^\top x \geq b \mid x \in \mathbb{S}_1\}$ and $\mathbb{S}_2$ on the other $\{a^\top x \leq b \mid x \in \mathbb{S}_2\}$.

Assuming the obstacles can be represented by the position of their four corners $\nu_{w,\iota}(t), \{\iota \in \mathbb{Z} \mid 1 \leq \iota \leq 4\}$, the collision avoidance constraint can be defined as:

$$
\begin{aligned}
a_{i,w}^\top(t)\nu_{w,\iota}(t) - b_{i,w}(t) \geq 0; \qquad & \forall \iota \in \mathbb{I}_1^4 \\
a_{i,w}^\top(t)x_i(t) - b_{i,w}(t) \leq -r_i^{\text{veh}} & \\
\|a_{i,w}(t)\|_2 \leq 1 & \\
\forall i \in \mathbb{I}_0^{N-1}; \forall w \in \mathbb{I}_0^{N_w - 1}, &
\end{aligned} \tag{11}
$$

where $a_{i,w}(t) \in \mathbb{R}^2$ and $b_{i,w}(t) \in \mathbb{R}$ represent the normal and offset vectors of the hyperplane associated with vehicle i and obstacle w. The normal and the offset vectors, alongside with the corners of the obstacles are all handled by B-spline trajectories $\hat{a}_{i,w}(t), \hat{b}_{i,w}(t)$ and $\hat{\nu}_{w,\iota}(t)$ respectively.

While in this work only static obstacles (w.r.t. $\mathcal{I}$) are considered, it has to be noted, that the method can be readily expanded to dynamic obstacles. By using (12) the positions of their corners can be transformed from $\mathcal{I}$ to $\mathcal{F}$ the same way, as it is done for static obstacles:

$$
\{\nu_{w,\iota}(t)\}_{\mathcal{F}} = {}^{\mathcal{F}}\mathbf{R}(\theta)_{\mathcal{I}} \cdot (\{\nu_{w,\iota}(t)\}_{\mathcal{I}} - r_{\text{f}}(t)), \tag{12}
$$

where ${}^{\mathcal{F}}\mathbf{R}(\theta)_{\mathcal{I}}$ is the matrix, which transforms a vector from the world coordinate frame $\mathcal{I}$ to the Frenet frame $\mathcal{F}$, and is defined as:

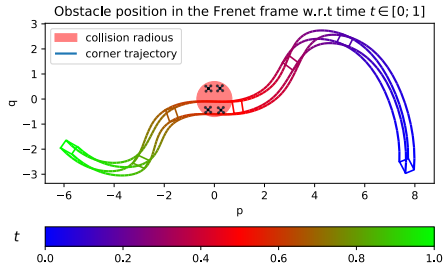


Fig. 2: Collision radious and the changing of the position of a static obstacle in the moving Frenet frame.

$$ {}^{\mathcal{F}}\mathbf{R}(\theta)_{\mathcal{I}} = \begin{bmatrix} \cos(\theta) & \sin(\theta) \\ -\sin(\theta) & \cos(\theta) \end{bmatrix}. \tag{13} $$

Constraints on the trajectories are enforced at t_s linearly spaced timesteps $t_s = \imath * T_s, \imath \in \mathcal{I}_0^{T_h/T_s}$, with T_h being the horizon length and T_s the sampling time.

F. Distributed optimization problem

A major drawback of centralised optimization techniques is, that for large problems they can be computationally expensive to solve. To circumvent this, it is often desirable to distribute them and solve the resulting, smaller problems on separate computational units.

In the centralised optimization problem (5) we can observe, that the objective function is composed of the individual objectives of the agents. Furthermore, all the constraints only apply to a single agent, except the equality constraints, which create a coupling between two agents. In order to distribute the optimization problem, this coupling between the agents has to be resolved.

The decoupling of the optimization problem is derived using the concept of the Alternating Direction Method of Multipliers (ADMM). As proposed by [15], to reduce the duality gap, only the coupling equality constraints are dualized. The derivation of the procedure is omitted for the sake of brevity, but the resulting algorithm will be presented in the next paragraph. The interested reader is however referred to [10] for a concise description of the required steps of the derivation.

By applying the ADMM concept to solve Problem (5), a distributed optimization problem is formulated and new decision variables are introduced. Each vehicle creates a copy of its own state trajectory and a copy of the neighboring vehicle's trajectory. The goal of the ADMM iterations is to equate the copies of the states with their real counterpart, while each adheres to its constraints and minimizes the respecting cost function. The duplicate of the ith state trajectory $\hat{x}_i(t)$ is represented by $\hat{z}_i(t)$, parameterised by ϑ_{z_i}. The copy of the jth state trajectory $\hat{x}_j(t)$, which is recommended by vehicle i to j, becomes $\hat{z}_{i\to j}(t)$ with coefficients $\vartheta_{\mathrm{z}_{i\to j}}$. Similarly, the copy of the ith vehicle, suggested by vehicle j to i is represented by $\hat{z}_{j\to i}(t)$, parameterised by $\vartheta_{\mathrm{z}_{j\to i}}$. Analogously we define $\hat{\phi}_i(t), \hat{\phi}_{i\to j}(t), \hat{\phi}_{j\to i}(t)$, parameterised by $\vartheta_{\phi_i}, \vartheta_{\phi_{i\to j}}, \vartheta_{\phi_{j\to i}}$ respectively. The ADMM concept also requires the creation of dual variables, associated with the dualised constraints. These will be indicated by the vectors λ_i, $\lambda_{i\to j}$, $\lambda_{j\to i}$ respectively. For simplification purposes, we combine all the coefficients into a single vector ϑ, whose fully updated form at the end of the ADMM iteration is denoted by ϑ^+.

Algorithm 1 summarizes the distributed ADMM update rule after the distribution of Problem (5). First, a *prediction* step is performed, whereby each agent plans a collision-free trajectory for itself, while obeying its system dynamics and minimizing the objective (6). Second, a *communication* step is performed, whereby agents share and receive these newly updated trajectories with their neighbours, so as each vehicle is aware of the generated trajectory of its neighbour. Third, in the *coordination* step, the duplicate variables ϑ_{z_i} and $\vartheta_{\mathrm{z}_{i\to j}}$ are updated. Essentially in this step trajectory suggestions are calculated for the individual vehicles, that adhere to the formation constraints. Fourth, the dual variables $\lambda_i, \lambda_{i\to j}$ are updated. Fifth, in the second *communication* step, the updated $\vartheta^+_{\mathrm{z}_{i\to j}}$ and $\lambda^+_{i\to j}$ values are shared with the neighbours so that each vehicle can integrate the trajectory suggestion of its neighbours in the next iteration. Lastly, all vectors are updated with the new vectors and the loop starts over.

G. Successive trajectory generation

The trajectories are computed piecewise, using a successive trajectory generation scheme. Algorithm 2 generates a new trajectory segment every Δt timestep for a horizon of T_h. More precisely, at every iteration l, where $l \in \mathbb{Z}_0^+$, a new trajectory $x_{i,l}(t)$ is generated for $[\Delta t \cdot l, \Delta t \cdot l + T_\mathrm{h}]$. The trajectory is passed to the local reference tracking controllers, assumed to be running on the vehicles. Each vehicle only evaluates the received trajectory for a length of Δt, because by the time it arrives to $x_{i,l}(\Delta t \cdot (l+1))$, a new trajectory is available for the $(l+1)$th iteration. Assuming perfect tracking, this point, and the starting point of the newly generated trajectory $x_{i,l+1}(\Delta t \cdot (l+1))$ overlap.

IV. Experimental results

In this section we show two examples. In both scenarios four vehicles have to keep a rectangular formation, while squeezing through a narrow corridor and avoid obstacles, that are placed on their path. The group has to follow the Frenet frame, which moves along a sinusoidal path with a constant speed. The entire environment is shown in Figure 3.

The simulations were implemented in Python. To formulate the optimization problem we have used CasADi [16] and IPOPT [17] to solve the non-linear optimization problem. The simulation was executed on a M1 Macbook Air, running MacOS 11.6.

A. Example 1: trajectory planning using receding horizon method

This first example demonstrates a scenario, where the trajectories are calculated iteratively using the successive

Algorithm 1 ADMM iteration performed by vehicle i.

initialize ϑ as zeros

repeat for n iterations

1. *Prediction*: update x_i

$$\begin{aligned}\vartheta_{\mathrm{x}_i}^{+} = \underset{\substack{\vartheta_{\mathrm{x}_i},\vartheta_{\mathrm{a}_{i,w}},\vartheta_{\mathrm{b}_{i,w}} \\ \forall w\in\mathbb{I}_0^{N_w-1}}}{\operatorname{argmin}} \quad & J_i(\vartheta_{\mathrm{x}_i}) \\ & + \lambda_i^\top(\vartheta_{\mathrm{x}_i}-\vartheta_{\mathrm{z}_i}) + \frac{\rho}{2}\|\vartheta_{\mathrm{x}_i}-\vartheta_{\mathrm{z}_i}\|_2^2 \\ & + \sum_{j\in\mathcal{N}_i}\left(\lambda_{j\to i}^\top(\vartheta_{\mathrm{x}_i}-\vartheta_{\mathrm{z}_{j\to i}}) + \frac{\rho}{2}\|\vartheta_{\mathrm{x}_i}-\vartheta_{\mathrm{z}_{j\to i}}\|_2^2\right) \\ \text{subject to} \quad & \vartheta_{\mathrm{x}_i}\in\mathcal{X}_i \\ & g_i^{\mathrm{way}}(\hat{x}_i(t_k)) \geq 0; \qquad \forall t_k\end{aligned}$$

2. *Communication* with agent j, $\forall j \in \mathcal{N}_i$

$$\begin{aligned}\text{send} \quad & \to \vartheta_{\mathrm{x}_i^+} \\ \text{receive} \quad & \leftarrow \{\vartheta_{\mathrm{x}_j^+}\}_{j\in\mathcal{N}_i}\end{aligned}$$

3. *Coordination*: updating ϑ_{z_i}, $\vartheta_{\mathrm{z}_{i\to j}}$

$$\begin{aligned}\begin{pmatrix}\vartheta_{\mathrm{z}_i}^{+} \\ \vartheta_{\mathrm{z}_{i\to j}}^{+}\end{pmatrix} = \underset{\substack{\vartheta_{\mathrm{z}_i},\vartheta_{\mathrm{z}_{i\to j}},\vartheta_{\phi_i} \\ \forall j\in\mathcal{N}_i}}{\operatorname{argmin}} \quad & \lambda_i^\top(\vartheta_{\mathrm{x}_i}-\vartheta_{\mathrm{z}_i}) \\ & + \frac{\rho}{2}\|\vartheta_{\mathrm{x}_i}-\vartheta_{\mathrm{z}_i}\|_2^2 \\ & + \sum_{j\in\mathcal{N}_i}(\lambda_{i\to j}^\top(\vartheta_{\mathrm{x}_j}-\vartheta_{\mathrm{z}_{i\to j}}) + \frac{\rho}{2}\|\vartheta_{\mathrm{x}_j}-\vartheta_{\mathrm{z}_{i\to j}}\|_2^2) \\ \text{subject to} \quad & g_i^{\mathrm{form}}(\hat{z}_i(t_s),\hat{z}_{i\to j}(t_s),\hat{\phi}_i(t_s)) = 0 \\ & g_i^{\mathrm{track}}(\hat{z}_i(t_s),\hat{z}_{i\to j}(t_s)) = 0 \\ & \vartheta_{\phi_i} = \vartheta_{\phi_j} \\ & \forall j\in\mathcal{N}_i; \forall t_s\end{aligned}$$

4. *Dual variable update*: computing λ_i^+, $\lambda_{i\to j}^+$

$$\begin{aligned}\lambda_i^+ &:= \lambda_i + \rho(\vartheta_{\mathrm{x}_i}^+ - \vartheta_{\mathrm{z}_i}^+) \\ \lambda_{i\to j}^+ &:= \lambda_{i\to j} + \rho(\vartheta_{\mathrm{x}_j}^+ - \vartheta_{\mathrm{z}_{i\to j}}^+)\end{aligned}$$

5. *Communication* with agent j, $\forall j \in \mathcal{N}_i$

$$\begin{aligned}\text{send} \quad & \to (\lambda_{i\to j}^+, \vartheta_{\mathrm{z}_{i\to j}}^+) \\ \text{receive} \quad & \leftarrow \{\lambda_{j\to i}^+, \vartheta_{\mathrm{z}_{j\to i}}^+\}_{\forall j\in\mathcal{N}_i}\end{aligned}$$

6. Update coefficients: $\vartheta \leftarrow \vartheta^+$

end repeat

return ϑ^+

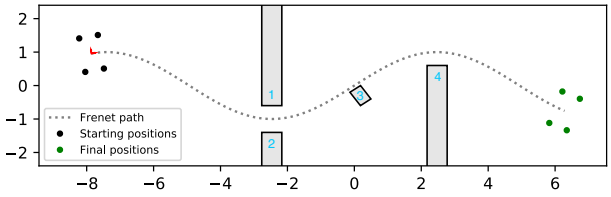


Fig. 3: Complex environment, with obstacles (grey boxes) and the path of the Frenet frame $\mathcal{F}$. Starting and final positions of the vehicles are denoted with black and green dots respectively.

Algorithm 2 Control algorithm for agent i

repeat every Δt, until final destination is reached

1. Update the list of obstacles and set $x_{i,l-1}(\Delta t \cdot l)$ as the new starting position.
2. Warm-start Algorithm 1 with the solution of the previous iteration.
3. Generate way-points for the interval $[\Delta t \cdot l, \Delta t \cdot l + T_{\mathrm{h}}]$ using the DFG algorithm.
4. Execute Algorithm 1 for n iterations.
5. Every agent i receives its updated trajectory segment $x_{i,l}(t)$ for the next time interval and starts following the new trajectory.

end repeat

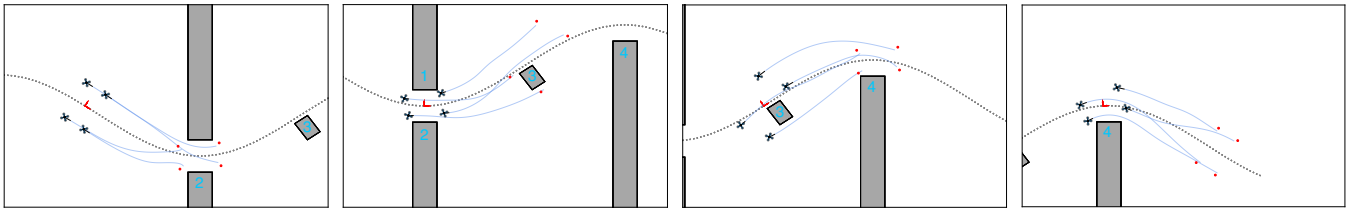
trajectory generation algorithm. The group is presented with a complex task. The goal is to keep a rectangular formation, while tracking the moving Frenet frame. Maintaining the shape however is only possible by rotation and scaling, due to the obstacles along the path.

Figure 4a shows DFG generating way-points (red dots), that urge the formation to turn. In Figure 4b we can observe the group passing the corridor and having planned trajectories, that rotate and increase the size of the formation to avoid the second obstacle. In Figure 4c the vehicles are avoiding obstacle 3 and are in the process of turning again to pass obstacle 4. Figure 4d shows the DFG algorithm generating way-points resembling the original formation, after T_{free} has passed and no new obstacle was detected.

Planning for a shorter horizon allows us to reduce the number of decision variables and the discrete timepoints, at which the constraints are enforced. For the example provided in Figure 4, splines with 5 coefficients were used. Formation and collision avoidance constraints were enforced at 10 timepoints. With a horizon length of 2 seconds (20% of the full 10 second long horizon) the average solution time for a single vehicle resulted in 0.3 seconds for a single ADMM iteration. Even for a worst-case scenario, new trajectories can be calculated every 0.4 seconds, resulting in a maximal update frequency of 2.5Hz.

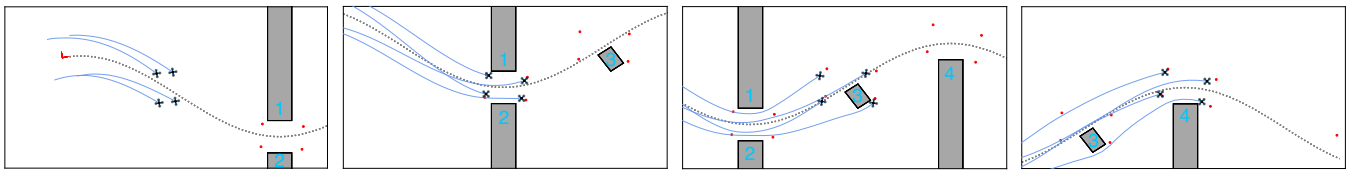
B. Example 2: trajectory planning for the entire path

Example 1 has showcased, that our algorithm is capable of guiding the group through a cluttered environment, without breaking the formation. Obstacle avoidance is achieved by only rotation and scaling of the formation. Dividing the entire trajectory into smaller sections however leads to sub-optimal performance, because only a part of the information is available to the solver. In contrast however, trying to solve a large non-linear optimization problem without proper initialization is likely to fail. By using DFG, the solver is able to find feasible trajectories for the group for the entire horizon. The generated motion trajectories shown in Figure 5 allow the group to perform a complex maneuver, consisting of fitting through a narrow corridor and avoiding obstacles along the path by only rotating and shrinking the formation.



(a) Vehicles approaching the gate. (b) Formation rotating and shrinking to fit through the corridor. (c) Group avoiding obstacle 3 by growing and rotating. (d) Formation turning to pass obstacle 4.

Fig. 4: Snapshots of Algorithm 2 planning new trajectories at every iteration, guided by way-points (red dots).



(a) Vehicles approaching the gate. (b) Formation rotating and shrinking to fit through the corridor. (c) Group avoiding obstacle 3 by growing and rotating. (d) Formation turning to pass obstacle 4.

Fig. 5: Snapshots of Algorithm 2 generating trajectories for the entire maneuver, guided by way-points (red dots).

In this example, generating way-points took 0.6 seconds. The trajectory consists of splines with a coefficient count of 28. The collision avoidance and formation constraints were enforced at 50 discrete timepoints. Using these parameters, we have achieved a mean solution time of 17 seconds for the entire procedure.

V. Conclusion and Future Work

In this work we have successfully demonstrated a novel, real-time algorithm, capable of navigating a group of vehicles through a complex environment.

The proposed hierarchical optimization scheme, which combines a lower and a higher level algorithm is capable of guiding the formation, while preserving the prescribed shape and only allowing scaling or rotation of the formation to avoid obstacles.

Future work include the implementation of the method on a physical system, in three dimensions and in a dynamic environment. Furthermore, the integration of the current method into a higher level trajectory planning algorithm is also being considered, where the Frenet path intelligently guides the vehicles, depending on the task at hand.

References

[1] D. Soetanto, L. Lapierre, and A. Pascoal, "Adaptive, Non-Singular Path-Following Control of Dynamic Wheeled Robots," in *42nd IEEE International Conference on Decision and Control*, (USA), pp. 1765–1770, 2003.

[2] A. P. Aguiar and J. P. Hespanha, "Trajectory-Tracking and Path-Following of Underactuated Autonomous Vehicles With Parametric Modeling Uncertainty," *IEEE Transactions on Automatic Control*, vol. 52, no. 8, pp. 1362–1379, 2007.

[3] K. Kanjanawanishkul, *Coordinated Path Following Control and Formation Control of Mobile Robots*. PhD thesis, Tübingen, 2010.

[4] C. W. Reynolds, "Flocks, herds and schools: A distributed behavioral model," in *Proceedings of the 14th Annual Conference on Computer Graphics and Interactive Techniques*, vol. 21, (New York, NY, USA), pp. 25–34, ACM Press, 1987.

[5] R. Cui, S. S. Ge, B. V. E. How, and Y. S. Choo, "Leader–follower formation control of underactuated autonomous underwater vehicles," *Ocean Engineering*, vol. 37, pp. 1491–1502, dec 2010.

[6] D. Zhou, Z. Wang, and M. Schwager, "Agile coordination and assistive collision avoidance for quadrotor swarms using virtual structures," *IEEE Transactions on Robotics*, vol. 34, pp. 916–923, aug 2018.

[7] F. Rey, Z. Pan, A. Hauswirth, and J. Lygeros, "Fully decentralized ADMM for coordination and collision avoidance," in *Proceedings of 2018 European Control Conference*, (Limassol, Cyprus), pp. 825–830, IEEE, jun 2018.

[8] Y. Lyu, J. Hu, B. M. Chen, C. Zhao, and Q. Pan, "Multivehicle flocking with collision avoidance via distributed model predictive control," *IEEE Transactions on Cybernetics*.

[9] R. Van Parys and G. Pipeleers, "Online distributed motion planning for multi-vehicle systems," in *Proceedings of 2016 European Control Conference*, (Aalborg, Denmark), pp. 1580–1585, IEEE, jun 2016.

[10] S. Novoth, Q. Zhang, K. Ji, and D. Yu, "Distributed Formation Control for Multi-Vehicle Systems With Splitting and Merging Capability," *IEEE Control Systems Letters*, vol. 5, no. 1, pp. 355–360, 2021.

[11] T. Mercy, W. Van Loock, and G. Pipeleers, "Real-time motion planning in the presence of moving obstacles," in *2016 European Control Conference, ECC 2016*, (Aalborg), pp. 1586–1591, IEEE, jun 2016.

[12] W. Van Loock, G. Pipeleers, and J. Swevers, "B-spline parameterized optimal motion trajectories for robotic systems with guaranteed constraint satisfaction," *Mechanical Sciences*, 2015.

[13] J. R. and C. de Boor, "A Practical Guide to Splines.," *Mathematics of Computation*, 2006.

[14] S. Boyd and L. Vandenberghe, *Convex Optimization*. Cambridge University Press, 2004.

[15] R. Raffard, C. Tomlin, and S. Boyd, "Distributed optimization for cooperative agents: application to formation flight," 43rd IEEE Conference on Decision and Control, 2004.

[16] J. Andersson, *A General-Purpose Software Framework for Dynamic Optimization*. PhD thesis, Katholieke Universiteit Leuven, Leuven, Belgium, 2013.

[17] A. Wächter and L. T. Biegler, "On the implementation of an interior-point filter line-search algorithm for large-scale nonlinear programming," *Mathematical Programming*, vol. 106, pp. 25–57, mar 2006.